

DD: OSR-RMA-01-FRAMEWORK — Equipment Swap & RMA Workflow Framework

Developer implementation guide: DD_OSR-RMA-01-FRAMEWORK-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_OSR-RMA-01-FRAMEWORK-v1.0.md.

Verdict: ■ New | **Wave:** 2 | **Group:** OSR (Operator Stock Registry) | **Version:** v1.0 | **Module:** Equipment Swap & RMA Workflow Framework **Satellite of:** OSR-RMA-01 Equipment Swap & RMA (parent) The shared Camunda + schema framework consumed by all four OSR-RMA-01 flow satellites (SWAP-HFC, SWAP-GPON, EQP Equipment Pickup, EQU Equipment Upgrade). Owns the equipment_swap_request root entity, the append-only history ledger, the shared workers and rules, the vendor-RMA stub for the deferred outbound-RMA flow, and the FOUNDATION_AUTH group codes the flows reuse for human tasks. v1.0 scope: framework primitives only. Per-flow BPMNs, per-flow REST endpoints, per-flow Kafka events, per-flow narrative are in the four satellites.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/08_osr_equipment/DD_OSR-RMA-01-FRAMEWORK-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /engine-rest/process-instance/{id}
- GET /api/swap-requests/{swap_request_id}
- GET /api/swap-requests/{swap_request_id}/history
- GET /api/swap-requests?operator=WIK&accountId=acct_01HZ_NJERI
- GET /api/swap-requests?operator=WIK¤tState=FAILED&since=2026-03-01

- GET /api/swap-requests?operator=WIK&recoveryContractorId=contractor_ke_nairobi_a
- GET /api/swap-requests?operator=WIK&swapKind=HFC_FIELD¤tState=AWAITING_SLOT
- PATCH /api/contractor-slot-commitments/{id}
- PATCH /api/homepasses/{id}/network-path
- PATCH /api/homepasses/{id}/network-path/...
- PATCH /api/homepasses/{id}/network-path/ports
- PATCH /api/vendor-rma-stubs/{id}
- PATCH /api/vendor-rma-stubs/{stub_id}
- PATCH /api/work-orders/{id}
- POST /api/billable-events
- POST /api/contractor-availability
- POST /api/contractor-slot-commitments
- POST /api/equipment-instances/inst_01HZ_CM_NJERI_OLD/lifecycle-events
- POST /api/equipment-instances/{id}/lifecycle-events
- POST /api/equipment-instances/{source_instance_id}/lifecycle-events
- POST /api/equipment-instances/{target_instance_id}/lifecycle-events
- POST /api/stock-movements
- POST /api/swap-requests/...
- POST /api/swap-requests/eqp
- POST /api/swap-requests/{id}/advance
- POST /api/swap-requests/{id}/cancel
- POST /api/swap-requests/{swap_request_id}/advance
- POST /api/swap-requests/{swap_request_id}/cancel
- POST /api/vendor-rma-stubs
- POST /api/work-orders
- POST /engine-rest/external-task/extt_01HZ_inst_state/complete
- POST /engine-rest/external-task/fetchAndLock

4.2 Tables

- equipment_swap_request
- equipment_swap_request_history
- vendor_rma_stub

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- assign-contractor
- bil01-charge-pre-authorize
- bpmn-cancel-requested

- contractor-slot-release
- contractor-slot-reserve
- contractor-stock-route
- customer-appointment-accepted
- customer-appointment-rejected
- dispatch-agent-wik-01
- eligibility-check
- field-techs
- finance-warranty-approvers
- homepass-network-path-update
- instance-lookup
- instance-state-transition
- nms-remote-bind-stub
- nms-remote-completed
- nms-remote-unassign-stub
- noc-agent-wik-01
- noc-confirm-remote-success
- noc-team
- notification-send
- payment-confirmed
- payment-timeout
- provisioning-adapter
- remote-attempt-timeout
- routing-decision-on-recovery
- routing-decision-recovered-instance-routes-to-recovering-contractor
- serial-rebind-broadhub-stub
- serial-unbind-broadhub-stub
- swap-dispatch-slot-selection
- swap-dispatchers
- swap-eligibility
- swap-request-history-append
- system-bpmn-engine
- tech-confirm-defect
- tech-confirm-recovery-or-eqr
- vendor-rma-coordinator-batch-ship
- vendor-rma-coordinators
- warranty-approver-decision
- wik-internal-tech-team
- wo-finalized

Worker-topic guidance:

- A BPMN service task uses the topic.

- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- EquipmentInstanceLifecycleTransition

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 2 | **Group:** OSR (Operator Stock Registry) | **Version:** v1.0 | **Module:** Equipment Swap & RMA Workflow Framework

Satellite of: OSR-RMA-01 Equipment Swap & RMA (parent)

The shared Camunda + schema framework consumed by all four OSR-RMA-01 flow satellites (SWAP-HFC, SWAP-GPON, EQP Equipment Pickup, EQU Equipment Upgrade). Owns the equipment_swap_request root entity, the append-only

history ledger, the shared workers and rules, the vendor-RMA stub for the deferred outbound-RMA flow, and the FOUNDATION_AUTH group codes the flows reuse for human tasks.

v1.0 scope: framework primitives only. Per-flow BPMNs, per-flow REST endpoints, per-flow Kafka events, per-flow narrative are in the four satellites.

1. What this module is

The four OSR-RMA-01 satellites all need the same plumbing: a swap-request root entity that tracks who/what/when, an append-only history of every state change, a set of Camunda service-task workers that operate on `equipment_instance` rows (per OSR-INSTANCE-01), a shared FOUNDATION_AUTH group set for dispatch / approval / vendor handoff, and a uniform way to fire Kafka events for downstream consumers. This DD is that shared layer.

The four flows are technology- and trigger-divergent but share 80% of the orchestration mechanics:

Flow	Trigger	Tech	Field visit?	Customer charge?
SWAP-HFC	Customer reports defect; or scheduled tech upgrade	HFC (DOCSIS cable modem ± MTA ± STB ± smart card)	Always — contractor brings new unit + retrieves defective	Free if under warranty / system fault; charge per BIL-01 catalog if customer-caused (negligence, water damage)
SWAP-GPON	Customer reports defect; or scheduled tech upgrade	GPON (ONT)	Conditional — try remote unassign + customer self-swap first; field fallback if customer can't or won't	Same as HFC
EQP (Equipment Pickup)	Subscription termination	Both technologies	Always — contractor retrieves end-of-life equipment	NA — no swap, just retrieval. EQR sub-state if customer refuses to return.
EQU (Equipment Upgrade)	Customer-requested or operator-initiated tech upgrade (DOCSIS 3.0 → 3.1, ONT v1 → v2, STB → STB-4K)	Both technologies	Always — contractor brings new unit, swaps, takes old	Per BIL-01 upgrade fee catalog; deposit delta may apply

What they all share:

1. A swap-request created in a DRAFT state.
2. Eligibility checks (warranty status, subscription state, customer billing status, slot/port availability).
3. Slot reservation against EM-02 contractor availability.
4. Source-instance lookup (the old device — for SWAP/EQP) and target-instance reservation (the new device — for SWAP/EQU).
5. Camunda BPMN that orchestrates field visit OR remote action OR mixed.
6. Lifecycle event recording on each `equipment_instance` involved (via OSR-INSTANCE-01 API).
7. Contractor-stock routing on recovered unit (THE Wananchi 40+ stale-equipment fix).
8. Cross-system provisioning calls: serial unbind/rebind in Broadhub for HFC; ONT remote unassign in NMS for GPON; subscription state update; billing trigger.
9. Customer + internal notifications.
10. Completion or failure recording in the history ledger; final state on the swap-request row.

The four satellites differ in: the BPMN process definition (different sequence of user tasks vs service tasks vs message catches), the eligibility rules, the provisioning calls (HFC = Broadhub double-config with slot/port preservation; GPON = NMS remote unassign with no slot/port preservation), and the customer-facing message templates.

What this module owns

1. `equipment_swap_request` — the workflow root entity. One row per swap request. Carries discriminator `swap_kind` to indicate which flow it belongs to.
2. `equipment_swap_request_history` — append-only ledger of every state change on a swap-request.

3. **vendor_rma_stub** — placeholder for the future outbound-RMA flow (defective unit → vendor → repaired → returned). v1.0 captures the handoff but no SLA tracking, no return-batch matching.
4. **Shared Camunda workers** (operator-agnostic Java beans, registered with the Camunda engine per FOUNDATION_CAMUNDA): instance-lookup, instance-state-transition, eligibility-check, contractor-slot-reserve, contractor-slot-release, contractor-stock-route, serial-unbind-broadhub-stub, serial-rebind-broadhub-stub, nms-remote-unassign-stub, nms-remote-bind-stub, swap-request-history-append, notification-send.
5. **Shared Drools rules**: swap-eligibility, routing-decision-on-recovery.
6. **Shared FOUNDATION_AUTH group codes used by flows**: swap-dispatchers, field-techs, noc-team, vendor-rma-coordinators, finance-warranty-approvers.
7. **Shared notification event kinds** consumed by NOT-01 / ICN-01 for swap-related notifications. OSR-RMA supplies the event kind and variables; NOT-01/ICN-01 own template selection, channel choice, language, quiet hours, and fallback.

What this module does NOT do

- **Doesn't define per-flow BPMN.** SWAP_HFC.bpmn, SWAP_GPON.bpmn, EQP.bpmn, EQU.bpmn live in the respective satellites. This DD only catalogs the workers + messages + user tasks they reuse.
- **Doesn't define per-flow REST endpoints.** Each satellite owns its POST /api/swap-requests/... shape with its own discriminator fields. The framework only declares the base equipment_swap_request schema; satellites extend via the JSONB flow_payload column.
- **Doesn't own equipment_instance.** Owned by OSR-INSTANCE-01. This DD's workers call OSR-INSTANCE-01 APIs.
- **Doesn't own stock movements.** Owned by OSR-01. When a swap recovers a unit and routes it back to a contractor van, the framework's contractor-stock-route worker calls OSR-01 POST /api/stock-movements AND OSR-INSTANCE-01 POST /api/equipment-instances/{id}/lifecycle-events in the same business transaction.
- **Doesn't perform vendor-side RMA workflows.** Outbound shipment to vendor, vendor repair SLA tracking, return-batch matching — all deferred. The stub captures the handoff timestamp + reference; that's all v1.0 provides.
- **Doesn't define charging rules.** Whether a swap is free (warranty / system fault) vs charged (customer-caused damage) is decided by BIL-01 charge codes referenced from the eligibility-check worker; not by this framework.
- **Doesn't decide tech-region routing.** The contractor for a given customer's swap is decided by reading account.tech_region_id (ILM-CFG-01) and looking up contractor_region_scope (EM-02). The framework calls EM-02's assign-contractor Drools rule; it does not redefine that logic.

Regional flexibility

KE is the v1.0 seed; schema designed for WUG/WTZ/YASSN variants:

- **Different tech mixes per operator.** WIK has both HFC + GPON; WUG/WTZ/YASSN may have GPON-only. Schema's swap_kind is operator-agnostic; satellites are deployed selectively per operator.
- **Different Broadhub / NMS versions.** WIK uses Broadhub with Clearcable NOMS for static-IP HFC; other operators have different OSS topologies. Workers like serial-unbind-broadhub-stub are STUBs in v1.0 — they call into an abstraction layer (provisioning-adaptor) per operator that's configured by ops, not hardcoded into the worker.
- **Different vendor RMA SLAs.** WIK uses Huawei + FiberHome (different SLA windows from Free Sénégal's vendors for YASSN). The vendor_rma_stub row carries vendor name + reference; per-vendor SLA is out of v1.0 scope.

Glossary

Term	Meaning
Swap-request	One row in equipment_swap_request representing one swap or pickup operation. The Camunda process instance is keyed to this row by its swap_request_id.
Source instance	The OSR-INSTANCE-01 equipment_instance being removed from the field (defective, end-of-life, or upgraded-from). NULL for installation-only flows (none in OSR-RMA-01, since install is FUL-02).
Target instance	The OSR-INSTANCE-01 equipment_instance being placed in the field (replacement, upgrade-to). NULL for EQP (pickup-only, nothing replacing).
Swap kind	Discriminator on equipment_swap_request: HFC_FIELD / GPON_REMOTE / GPON_FIELD / EQP_TERMINATION / EQU_UPGRADE. Each maps to one BPMN.

Term	Meaning
Recovery contractor	The EM-02 contractor responsible for physically retrieving the source instance from the field. Tracked on the swap-request AND on the <code>equipment_instance_lifecycle_event</code> <code>CONTRACTOR_RECOVERED_FROM_FIELD</code> row. This is the contractor whose warehouse the recovered unit routes back to.
EQR sub-state	"Equipment Recovery failed" — customer refused to return the equipment. Closes the EQP swap-request with <code>final_state=COMPLETED_WITHOUT_RECOVERY</code> . Subscription closure still proceeds; financial impact via BIL-01 deposit forfeiture and/or write-off.
Outbound RMA	The vendor side of the RMA flow (defective unit → vendor → repair → return). v1.0 STUB only.
Slot/port preservation	HFC-specific constraint: the new modem MUST be authorized on the same CMTS slot/port as the old one (UC-PR4, KE workshop). Enforced by SWAP-HFC satellite's eligibility check and by <code>serial-rebind-broadhub-stub</code> payload.
Provisioning adapter	An operator-specific abstraction over Broadhub / NMS / VoipSwitch / Verimatrix that the framework's workers call. Each *-stub worker calls the adapter; the adapter resolves to the real OSS call per operator config. v1.0 ships with a no-op default adapter for environments where the real OSS isn't connected; ops switch the active adapter via config.

2. Tables overview + relationships

2.1 Tables overview

OSR-RMA-01-FRAMEWORK owns three tables:

Table	Purpose
<code>equipment_swap_request</code>	The workflow root entity. One row per swap request. Carries <code>swap_kind</code> discriminator.
<code>equipment_swap_request_history</code>	Append-only ledger of every state change on a swap-request.
<code>vendor_rma_stub</code>	Placeholder for the future outbound-RMA flow. v1.0 captures handoff metadata only.

Full DDL in §3.

2.2 Entity relationships

[Missing image: file:///home/claude/diagrams/osr_rma_framework_erd.png]

Reading the diagram:

- `equipment_swap_request` is the root. One row per swap operation.
- `equipment_swap_request_history` is one-to-many off the swap-request — every state change is one row.
- `equipment_instance` (OSR-INSTANCE-01) is referenced in two slots: `source_instance_id` (the device being removed from field) and `target_instance_id` (the device being placed in field). Both nullable: EQP has no target; install-only flows would have no source (but those live in FUL-02, not here).
- `account` (ILM-CFG-01) is referenced by `account_id` — the customer affected by the swap.
- `subscription` (ILM-CFG-01) is referenced by `subscription_id` — usually 1:1 with `account` in v1.0 (per ILM-CFG-01 design), but kept as a separate FK for forward compatibility.
- `contractor` (EM-02) is referenced by `recovery_contractor_id` — who physically handles the swap. THIS IS THE FIELD that determines where the recovered unit routes back to.
- `contractor_slot_commitment` (EM-02) is referenced by `slot_commitment_id` — the slot reservation against contractor availability.
- `wo` (WO-01) is referenced by `wo_id` — the work order created to dispatch the field visit (NULL for purely remote flows like SWAP-GPON happy path).
- `vendor_rma_stub` is referenced by `outbound_rma_stub_id` when the source instance is shipped to vendor for repair (set later, may be NULL at creation).

2.3 Cross-DD anchoring

- **OSR-INSTANCE-01 Equipment Instance Registry** — the framework's workers (instance-lookup, instance-state-transition, contractor-stock-route) operate on equipment_instance rows by calling OSR-INSTANCE-01 APIs. Every swap step that changes an instance's state emits one EquipmentInstanceLifecycleTransition event via OSR-INSTANCE-01.
- **OSR-01 Stock Chain Engine** — when a swap recovers a unit and routes it to a contractor warehouse, contractor-stock-route calls OSR-01 POST /api/stock-movements (for the SKU+qty rollup) AND OSR-INSTANCE-01 POST /api/equipment-instances/{id}/lifecycle-events (for the per-serial detail) in the same transaction.
- **EM-02 Contractor & Staff Registry** — the framework calls EM-02's POST /api/contractor-availability (to find eligible contractor) and POST /api/contractor-slot-commitments (to reserve a slot). Slot release on swap-request cancellation calls EM-02's PATCH /api/contractor-slot-commitments/{id} with status=RELEASED.
- **WO-01 Work Order Module** — flows that require a field visit (SWAP-HFC, SWAP-GPON field-fallback, EQP, EQU) create a WO via POST /api/work-orders with wo_kind=EQUIPMENT_SWAP / EQUIPMENT_PICKUP / EQUIPMENT_UPGRADE. The WO carries swap_request_id in its origin_ref. WO finalization fires a message event back into the swap-request BPMN.
- **ILM-CFG-01 Customer Service Catalog** — the framework reads account.account_id, account.tech_region_id, account.payment_account_number, subscription.subscription_id to identify the customer + drive tech-region-based contractor selection. Account is the binding anchor (Customer is identity-only).
- **PLM-CFG-06 Equipment Type Catalog** — the framework validates that the source and target instances reference SKUs of compatible equipment_kind (e.g., source ONT can be replaced by target ONT but not by target STB). Eligibility worker reads PLM-CFG-06.
- **BIL-01 Charging Engine** — eligibility-check worker calls BIL-01 to (a) check the customer's billing state (suspended? in dunning?), (b) determine if the swap is chargeable (warranty status + reason code → charge code mapping), (c) trigger the charge on completion if applicable.
- **NOT-01 Customer Notifications** — flows send customer-facing notifications (swap appointment confirmation, technician en route, completion confirmation, EQR equipment-return-required reminder) via NOT-01.
- **ICN-01 Internal Communications** — flows fan out notifications to internal candidate groups: swap dispatchers, field techs, NOC team, vendor RMA coordinators, finance warranty approvers.

2.4 Satellite consumption

Satellite	Uses framework's	Doesn't use
OSR-RMA-01-FLOW-SWAP-HFC	equipment_swap_request (kind=HFC_FIELD), all workers, history ledger, all FOUNDATION_AUTH groups	nms-remote-unassign-stub, nms-remote-bind-stub (HFC uses Broadhub adapters instead)
OSR-RMA-01-FLOW-SWAP-GPON	equipment_swap_request (kind=GPON_REMOTE OR GPON_FIELD), all workers, history ledger	serial-unbind-broadhub-stub / serial-rebind-broadhub-stub (GPON uses NMS instead)
OSR-RMA-01-FLOW-EQP	equipment_swap_request (kind=EQP_TERMINATION), most workers, history ledger, EQR sub-state handling	serial-rebind-broadhub-stub, nms-remote-bind-stub (no replacement device)
OSR-RMA-01-FLOW-EQU	equipment_swap_request (kind=EQU_UPGRADE), all workers including chargeable-upgrade path	Optional vendor-RMA stub (upgrades typically don't trigger RMA — the old unit goes back to stock for reuse, not vendor)

3. Schemas

3.1 equipment_swap_request

```

CREATE TABLE equipment_swap_request (
  swap_request_id      TEXT PRIMARY KEY,          -- ULID 'swap_01HZ...'
  operator_code        TEXT NOT NULL,             -- 'WIK' | 'WUG' | 'WTZ' | 'YASSN'
  swap_kind            TEXT NOT NULL,             -- discriminator; one of the 5 values below
  current_state        TEXT NOT NULL,             -- workflow state; see state machine §4.1
  account_id           TEXT NOT NULL REFERENCES account(account_id),
  subscription_id      TEXT NULL REFERENCES subscription(subscription_id),
  source_instance_id   TEXT NULL REFERENCES equipment_instance(instance_id), -- NULL for install-only (not applicable)
)
```



```

target_instance_id    TEXT NULL REFERENCES equipment_instance(instance_id), -- NULL for EQP (pickup only)
reason_code           TEXT NOT NULL, -- 'CUSTOMER_REPORTED_DEFECT','SCHEDULED_UPGRADE','TERMINATION'
initiated_by_user_id  TEXT NOT NULL, -- FOUNDATION_AUTH user who created the request
initiated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
recovery_contractor_id TEXT NULL, -- EM-02 contractor_id; set after eligibility + assignment; THE
slot_commitment_id    TEXT NULL, -- EM-02 contractor_slot_commitment.commitment_id; set after s
wo_id                 TEXT NULL, -- WO-01 wo_id; set when a field WO is created; NULL for pure r
outbound_rma_stub_id  TEXT NULL REFERENCES vendor_rma_stub(stub_id), -- set when the source instance is shipped to ve
charge_code           TEXT NULL, -- BIL-01 charge code applied; resolved at eligibility for char
customer_charge_amount NUMERIC(14,4) NULL, -- resolved at eligibility (chargeable swaps) or at completion
chargeable            BOOLEAN NOT NULL DEFAULT FALSE, -- TRUE iff the swap requires customer payment before dispatch
billable_event_id     TEXT NULL, -- BIL-01 billable_event id when pre-authorize charge created;
payment_received_at   TIMESTAMP NULL, -- timestamp when BIL-01 confirms the customer wallet covers th
flow_payload          JSONB NULL, -- per-flow discriminated fields: HFC carries {slot,port,authz
bpmn_process_instance_id TEXT NULL, -- Camunda process instance ID; set after BPMN start
final_state           TEXT NULL, -- terminal state when swap closes; NULL while in flight
closed_at            TIMESTAMP NULL,
notes                TEXT NULL,
created_at           TIMESTAMP NOT NULL DEFAULT NOW(),
updated_at           TIMESTAMP NOT NULL DEFAULT NOW(),
CHECK (swap_kind IN ('HFC_FIELD','GPON_REMOTE','GPON_FIELD','EQP_TERMINATION','EQU_UPGRADE')),
CHECK (current_state IN (
'DRAFT','ELIGIBILITY_CHECK','AWAITING_PAYMENT','AWAITING_SLOT','SLOT_RESERVED','WO_CREATED','FIELD_VISIT_IN_PROGRESS',
'REMOTE_UNASSIGN_IN_PROGRESS','SOURCE_RECOVERED','TARGET_PLACED','PROVISIONING_IN_PROGRESS',
'COMPLETED','CANCELLED','FAILED','COMPLETED_WITHOUT_RECOVERY'
)),
CHECK (final_state IS NULL OR final_state IN ('COMPLETED','CANCELLED','FAILED','COMPLETED_WITHOUT_RECOVERY')),
CHECK (reason_code IN ('CUSTOMER_REPORTED_DEFECT','SCHEDULED_UPGRADE','TERMINATION','SYSTEM_FAULT_PROACTIVE','CUSTOMER_F
));

CREATE INDEX idx_swap_request_account ON equipment_swap_request (account_id, current_state);
CREATE INDEX idx_swap_request_source ON equipment_swap_request (source_instance_id) WHERE source_instance_id IS NOT
CREATE INDEX idx_swap_request_target ON equipment_swap_request (target_instance_id) WHERE target_instance_id IS NOT
CREATE INDEX idx_swap_request_contractor ON equipment_swap_request (recovery_contractor_id, current_state) WHERE recove
CREATE INDEX idx_swap_request_wo ON equipment_swap_request (wo_id) WHERE wo_id IS NOT NULL;
CREATE INDEX idx_swap_request_state_open ON equipment_swap_request (operator_code, current_state) WHERE final_state IS
CREATE INDEX idx_swap_request_bpmn ON equipment_swap_request (bpmn_process_instance_id) WHERE bpmn_process_instan

```

Sample data:

swap_reque st_id	operator_co de	swap_kind	current_stat e	account_id	source_inst ance_id	target_insta nce_id	reason_cod e	recovery_co ntractor_id	wo_id	final_state
swap_01HZ _NJERI_MO DEM	WIK	HFC_FIELD	COMPLETED	acct_01HZ _NJERI	inst_01HZ _CM_NJERI _OLD	inst_01HZ _CM_NJERI _NEW	CUSTOMER_ REPORTED_ DEFECT	contracto r_ke_nair obi_a	wo_01HZ_N JERI_SWAP	COMPLETED
swap_01HZ _WANJIKU_ ONT	WIK	GPON_REMO TE	COMPLETED	acct_01HZ _WANJIKU	inst_01HZ _ONT_WANJ IKU_OLD	inst_01HZ _ONT_WANJ IKU_NEW	CUSTOMER_ REPORTED_ DEFECT	wik-inter nal-tech- team	NULL	COMPLETED
swap_01HZ _KAMAU_TE RM	WIK	EQP_TERMI NATION	COMPLETED	acct_01HZ _KAMAU	inst_01HZ _ONT_KAMA U	NULL	TERMINATI ON	contracto r_ke_nair obi_a	wo_01HZ_K AMAU_EQP	COMPLETED
swap_01HZ _OTIENO_T ERM_EQR	WIK	EQP_TERMI NATION	COMPLETED _WITHOUT_ RECOVERY	acct_01HZ _OTIENO	inst_01HZ _STB_OTIE NO	NULL	TERMINATI ON	contracto r_ke_nair obi_a	wo_01HZ_O TIENO_EQP	COMPLETED _WITHOUT_ RECOVERY
swap_01HZ _MUTHONI_ UPG	WIK	EQU_UPGRA DE	FIELD_VIS IT_IN_PRO GRESS	acct_01HZ _MUTHONI	inst_01HZ _CM_MUTHO NI_DOCSIS 30	inst_01HZ _CM_MUTHO NI_DOCSIS 31	CUSTOMER_ REQUESTED _TECH_CHA NGE	contracto r_ke_nair obi_a	wo_01HZ_M UTHONI_UP G	NULL
swap_01HZ _DKR_ONT_ DEFECT	YASSN	GPON_FIEL D	AWAITING_ SLOT	acct_01HZ _DKR_001	inst_01HZ _DKR_ONT_ DEFECT_01	NULL	CUSTOMER_ REPORTED_ DEFECT	NULL	NULL	NULL
swap_01HZ _PROACTIV E_BATCH_0 5	WIK	HFC_FIELD	SLOT_RESE RVED	acct_01HZ _PROACTIV E_05	inst_01HZ _CM_PROAC TIVE_05	inst_01HZ _CM_REPLA CEMENT_05	SYSTEM_FA ULT_PROAC TIVE	contracto r_ke_nair obi_b	NULL	NULL
swap_01HZ _FAILED_N RB_01	WIK	GPON_REMO TE	FAILED	acct_01HZ _FAILED_0 1	inst_01HZ _ONT_FAIL ED_01	inst_01HZ _ONT_REPL ACEMENT_0 1	CUSTOMER_ REPORTED_ DEFECT	NULL	NULL	FAILED

Row 1: Njeri's HFC cable modem failed in the field. Reason = CUSTOMER_REPORTED_DEFECT. Source = her old modem, target = a replacement modem (inst_01HZ_CM_NJERI_NEW was previously in main warehouse, dispatched to Franchise A's van for this WO). Field visit completed; recovery_contractor_id = Franchise A; WO created and finalized; both instances transitioned through OSR-INSTANCE-01 lifecycle. State = COMPLETED.

Row 2: Wanjiku's GPON ONT was swapped remotely — the customer was technically capable, so the WIK internal tech team coordinated by phone, the old ONT was unassigned in NMS, the new ONT (which Wanjiku already had as part of a self-install kit shipped earlier) was bound. No wo_id because no field visit. recovery_contractor_id = WIK Internal Tech Team — they're the contractor on record even though the work was remote.

Row 3: Kamau terminated his subscription. EQP swap created; Franchise A dispatched to retrieve his ONT. Source = the ONT at Kamau's premises. No target (this is a pickup, not a swap). WO finalized; equipment recovered; current_state = COMPLETED.

Row 4: Otieno also terminated, BUT he refused to return his STB. EQP swap-request created the same way; field visit occurred; the contractor noted "customer refused return" on the WO; current_state = COMPLETED_WITHOUT_RECOVERY and final_state = COMPLETED_WITHOUT_RECOVERY (the EQR sub-state). BIL-01 will trigger deposit forfeiture / write-off processing.

Row 5: Muthoni is upgrading from a DOCSIS 3.0 cable modem to a DOCSIS 3.1 modem. EQU_UPGRADE. Field visit currently in progress. charge_code not yet set (charge applies on completion per BIL-01 upgrade-fee catalog). flow_payload carries upgrade-from-SKU and upgrade-to-SKU plus the prorated fee.

Row 6: YASSN-side ONT defect, waiting for a slot to be reserved. recovery_contractor_id NULL because the slot hasn't been reserved yet (the slot reservation operation atomically sets both fields). swap_kind = GPON_FIELD because — let's say — YASSN doesn't run a remote-self-swap workflow for this customer tier; field visit is mandated.

Row 7: A proactive system-detected fault. NOC flagged the modem as showing signs of imminent failure; SWAP created with reason_code = SYSTEM_FAULT_PROACTIVE. Slot reserved with contractor_ke_nairobi_b. Awaiting WO creation.

Row 8: A failed swap. Maybe the remote NMS call failed three times, or the customer never accepted the appointment, or the eligibility check ultimately failed after retries. current_state and final_state both FAILED. BPMN process terminated. Reason captured in history ledger (§3.2).

3.2 equipment_swap_request_history

```
CREATE TABLE equipment_swap_request_history (
  history_id          TEXT PRIMARY KEY,          -- ULID 'swh_01HZ...'
  swap_request_id     TEXT NOT NULL REFERENCES equipment_swap_request(swap_request_id),
  event_sequence      BIGINT NOT NULL,          -- monotonic per swap-request
  from_state          TEXT NULL,                -- NULL for the first row (creation)
  to_state             TEXT NOT NULL,
  transition_reason    TEXT NOT NULL,          -- e.g., 'CREATED', 'ELIGIBILITY_PASSED', 'SLOT_RESERVED', 'WO_FINALIZED'
  actor_user_id       TEXT NOT NULL,          -- FOUNDATION_AUTH user (or system identity) responsible for transition
  bpmn_task_id        TEXT NULL,              -- Camunda task ID when the transition came from a BPMN task
  worker_name         TEXT NULL,              -- e.g., 'eligibility-check', 'contractor-slot-reserve'; NULL for hub
  payload             JSONB NULL,             -- worker-specific output: {'reservedSlotId':...} or {'broadhubAdaptation':...}
  notes               TEXT NULL,
  occurred_at         TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX uq_swap_history_seq
  ON equipment_swap_request_history (swap_request_id, event_sequence);
CREATE INDEX idx_swap_history_by_swap
  ON equipment_swap_request_history (swap_request_id, occurred_at DESC);
CREATE INDEX idx_swap_history_by_reason
  ON equipment_swap_request_history (transition_reason, occurred_at DESC);
CREATE INDEX idx_swap_history_by_worker
  ON equipment_swap_request_history (worker_name, occurred_at DESC) WHERE worker_name IS NOT NULL;
```

Sample data (full history for Row 1 above — Njeri's HFC modem swap):

history_id	swap_request_id	event_sequence	from_state	to_state	transition_reason	actor_user_id	worker_name	occurred_at
swh_01HZ_E1	swap_01HZ_NJERI_MODEM	1	NULL	DRAFT	CREATED	noc-agent-wik-01	NULL	2026-03-10T09:00:00+03:00
swh_01HZ_E2	swap_01HZ_NJERI_MODEM	2	DRAFT	ELIGIBILITY_CHECK	BPMN_STARTED	system-bpmn-engine	NULL	2026-03-10T09:00:01+03:00
swh_01HZ_E3	swap_01HZ_NJERI_MODEM	3	ELIGIBILITY_CHECK	AWAITING_SLOT	ELIGIBILITY_PASSED	system-bpmn-engine	eligibility-check	2026-03-10T09:00:03+03:00
swh_01HZ_E4	swap_01HZ_NJERI_MODEM	4	AWAITING_SLOT	SLOT_RESERVED	SLOT_RESERVED	dispatch-agent-wik-01	contractor-slot-reserve	2026-03-10T10:15:00+03:00

history_id	swap_request_id	event_sequence	from_state	to_state	transition_reason	actor_user_id	worker_name	occurred_at
swh_01HZ_E5	swap_01HZ_NJERI_MODEM	5	SLOT_RESERVED	WO_CREATED	WO_CREATED	dispatch-agent-wik-01	NULL	2026-03-10T10:15:01+03:00
swh_01HZ_E6	swap_01HZ_NJERI_MODEM	6	WO_CREATED	FIELD_VISIT_IN_PROGRESS	WO_DISPATCHED	dispatch-agent-wik-01	NULL	2026-03-11T08:30:00+03:00
swh_01HZ_E7	swap_01HZ_NJERI_MODEM	7	FIELD_VISIT_IN_PROGRESS	SOURCE_RECOVERED	TECH_RECOVERED_SOURCE	tech_franchise_a_03	instance-state-transition	2026-03-11T11:20:00+03:00
swh_01HZ_E8	swap_01HZ_NJERI_MODEM	8	SOURCE_RECOVERED	TARGET_PLACED	TECH_PLACED_TARGET	tech_franchise_a_03	instance-state-transition	2026-03-11T11:35:00+03:00
swh_01HZ_E9	swap_01HZ_NJERI_MODEM	9	TARGET_PLACED	PROVISIONING_IN_PROGRESS	STARTING_BROADHUB_REBIND	system-bpmn-engine	serial-rebind-broadhub-stub	2026-03-11T11:35:02+03:00
swh_01HZ_E10	swap_01HZ_NJERI_MODEM	10	PROVISIONING_IN_PROGRESS	COMPLETED	BROADHUB_REBIND_OK	system-bpmn-engine	serial-rebind-broadhub-stub	2026-03-11T11:35:08+03:00

Row 1: NOC agent creates the swap-request after Njeri calls customer care reporting intermittent connection. State = DRAFT.

Row 2: BPMN engine picks up the new request and transitions to ELIGIBILITY_CHECK. System actor.

Row 3: eligibility-check worker runs. Validates warranty status (Njeri's modem is 6 months old, within 18-month warranty), customer billing status (account active, not in dunning), tech-region availability (Franchise A serves Njeri's NRB-KAREN region). Eligibility passed. State → AWAITING_SLOT.

Row 4: Dispatch agent reviews pending swaps and reserves a slot for tomorrow at 11:00 in Franchise A's morning slot. recovery_contractor_id = contractor_ke_nairobi_a, slot_commitment_id populated. State → SLOT_RESERVED.

Row 5: WO created via WO-01 with wo_kind=EQUIPMENT_SWAP and swap_request_id in origin_ref. State → WO_CREATED. wo_id populated on the swap-request row.

Row 6: Next morning, dispatch dispatches the WO to the technician. State → FIELD_VISIT_IN_PROGRESS.

Row 7: Tech arrives, confirms defect, physically removes the old modem from Njeri's premises. instance-state-transition worker called: source instance transitions IN_FIELD_DEFECTIVE → RECOVERED_BY_CONTRACTOR, customer binding cleared, current_location_id set to Franchise A's van, contractor_id recorded on the lifecycle event. State on swap-request → SOURCE_RECOVERED.

Row 8: Tech installs the new modem. instance-state-transition: target instance RESERVED_FOR_WO → IN_FIELD_ACTIVE, customer binding established. State → TARGET_PLACED.

Row 9-10: serial-rebind-broadhub-stub worker called to update Broadhub: unbind the old CM-MAC, bind the new CM-MAC on the SAME slot/port (UC-PR4 — slot/port preservation enforced). Adapter returns OK. State → PROVISIONING_IN_PROGRESS → COMPLETED.

3.3 vendor_rma_stub

v1.0 STUB for the deferred outbound-RMA workflow. Captures handoff metadata so the source-instance lifecycle can transition to RMA_SHIPPED_TO_VENDOR with traceability, but no SLA tracking, no return-batch matching, no vendor-portal integration.

```
CREATE TABLE vendor_rma_stub (
  stub_id          TEXT PRIMARY KEY,
  operator_code    TEXT NOT NULL,
  vendor_name      TEXT NOT NULL,
  vendor_ref       TEXT NULL,
  shipped_at        TIMESTAMP NULL,
  expected_return_at DATE NULL,
  returned_at      TIMESTAMP NULL,
  return_outcome   TEXT NULL,
  vendor_notes     TEXT NULL,
  initiated_by_user_id TEXT NOT NULL,
  created_at       TIMESTAMP NOT NULL DEFAULT NOW(),
  CHECK (return_outcome IS NULL OR return_outcome IN ('REPAIRED', 'REPLACED', 'BEYOND_REPAIR', 'LOST_BY_VENDOR'))
);
```

```
CREATE INDEX idx_vendor_rma_stub_in_flight
ON vendor_rma_stub (operator_code, vendor_name) WHERE returned_at IS NULL;
CREATE INDEX idx_vendor_rma_stub_by_vendor_ref
ON vendor_rma_stub (vendor_name, vendor_ref) WHERE vendor_ref IS NOT NULL;
```

Sample data:

stub_id	operator_code	vendor_name	vendor_ref	shipped_at	expected_return_at	returned_at	return_outcome
rma_01HZ_BATCH_HUAWEI_05	WIK	Huawei	HUA-RMA-2026-0234	2026-03-15T10:00:00+03:00	2026-04-15	NULL	NULL
rma_01HZ_BATCH_FIBERHOME_02	WIK	FiberHome	FH-RMA-2026-0098	2026-02-20T14:30:00+03:00	2026-03-20	2026-03-25T09:00:00+03:00	REPAIRED
rma_01HZ_BATCH_CISCO_01	WIK	Cisco	NULL	2026-03-18T11:00:00+03:00	2026-04-18	NULL	NULL
rma_01HZ_BATCH_YASSN_HUAWEI_06	YASSN	Huawei	HUA-RMA-2026-DKR-005	2026-02-10T16:00:00+00:00	2026-03-10	2026-03-08T14:00:00+00:00	BEYOND_REPAIR

Row 1: 50 defective ONTs batched and shipped to Huawei. Vendor reference issued. Expected return in 30 days. In flight.

Row 2: A FiberHome batch shipped late February, returned March 25 (5 days late). Vendor returned the units repaired.

Row 3: A Cisco shipment without a vendor reference yet (vendor hasn't issued one, or ops hasn't received it back). Stub still useful for tracking the source-instance state.

Row 4: A YASSN Huawei batch; one or more units declared beyond-repair by vendor. The outbound-RMA stub captures the outcome; the affected equipment_instance rows downstream are transitioned RMA_SHIPPED_TO_VENDOR → DECOMMISSIONED instead of → REPAIRED_BY_VENDOR.

In v1.0, the stub is populated by ops via a back-office screen when they ship a batch and when they receive a return batch; it's not driven by vendor portal integration. Future expansion will replace this with a real vendor-RMA module.

4. Rules

4.1 State machine for equipment_swap_request

Legal transitions (rows = from, columns = to). Terminal states: COMPLETED, CANCELLED, FAILED, COMPLETED_WITHOUT_RECOVERY.

From \ To	DRAFT	ELIGIBILITY_CHECK	AWAITING_PAYMENT	AWAITING_SLOT	SLOT_RESERVED	WORK_CREATED	FIELD_VISIT	REMOTE_UNASS	SOURCE_RECOV	TARGET_PLACED	PROVIN_PR OG	COMPLETED	CANCELLED	FAILED	COMPLETION_NO_RECOVERY
---	⌵	⌵	⌵	⌵	⌵	⌵	⌵	⌵	⌵	⌵	⌵	⌵	⌵	⌵	⌵
(init)	✓														
DRAFT		✓											✓		
ELIGIBILITY_CHECK			✓ (chargeable)	✓ (free)									✓	✓	
AWAITING_PAYMENT				✓ (paid)									✓	✓	
AWAITING_SLOT					✓								✓	✓	
SLOT_RESERVED				✓ (release)		✓		✓					✓	✓	
WORK_CREATED							✓						✓	✓	

From \ To	DRAFT	ELIG_CHECK	AWAITING_PAYMENT	AWAITING_SLOT	SLOT_RESERVED	WO_CREATED	FIELD_VISIT	REMOTE_UNASSIGN	SOURCE_RECOVERED	TARGET_PLACED	PROVISIONING	COMPLETED	CANCELLED	FAILED	COMPLETED_WITHOUT_RECOVERY
FIELD_VISIT_IN_PROGRESS								✓					✓	✓	✓ (EQP only)
REMOTE_UNASSIGN_IN_PROGRESS								✓					✓	✓	
SOURCE_RECOVERED									✓	✓ (EQP/EQR direct)	✓ (EQP/EQR direct)	✓	✓	✓	✓
TARGET_PLACED										✓				✓	
PROVISIONING_IN_PROGRESS											✓			✓	
COMPLETED												termina l			
CANCELLED													termina l		
FAILED														termina l	
COMPLETED_WITHOUT_RECOVERY															termina l

Notes:

- **AWAITING_PAYMENT is the pay-first gate for chargeable swaps** (workshop 2026-04-28 wallet funding gate). Free swaps (under warranty, system fault, operator-paid like Shifting) skip this state and go ELIGIBILITY_CHECK → AWAITING_SLOT directly. Chargeable swaps go ELIGIBILITY_CHECK → AWAITING_PAYMENT → AWAITING_SLOT only after BIL-01 confirms payment. No slot is reserved before payment is confirmed; no contractor is dispatched; no truck rolls. Payment timeout (default 7 days) cancels the swap-request.
- REMOTE_UNASSIGN_IN_PROGRESS is reached only by GPON_REMOTE (no field visit). Successful remote unassign + customer self-swap path: REMOTE_UNASSIGN_IN_PROGRESS → SOURCE_RECOVERED → PROVISIONING_IN_PROGRESS (remote-bind-new) → COMPLETED. Failed remote path falls back to creating a WO; transition is REMOTE_UNASSIGN_IN_PROGRESS → SLOT_RESERVED → WO_CREATED → FIELD_VISIT_IN_PROGRESS → . . . Or, alternative: REMOTE_UNASSIGN_IN_PROGRESS → FAILED if no field fallback is configured for that operator.
- EQP flow skips TARGET_PLACED (no replacement device): SOURCE_RECOVERED → PROVISIONING_IN_PROGRESS (subscription closure, BIL-01 deposit refund) → COMPLETED.
- EQR sub-state (customer refuses return): FIELD_VISIT_IN_PROGRESS → COMPLETED_WITHOUT_RECOVERY directly. The source instance stays in IN_FIELD_DEFECTIVE or transitions to DECOMMISSIONED per BIL-01 forfeiture path.
- SLOT_RESERVED → AWAITING_SLOT is the slot-release path: when a swap is rescheduled, the original slot is released and a new slot is requested.
- **HomePass network_path update is a side effect of TARGET_PLACED for GPON swaps only.** See R-OSR-RMA-16. The homepass-network-path-update worker runs synchronously on the transition; its failure aborts the swap to FAILED.

4.2 Rules

Rule	Description
R-OSR-RMA-1	Exactly one open swap-request per (account_id, source_instance_id) — concurrent swap creation for the same source instance is rejected with HTTP 409 + code=ALREADY_IN_SWAP. Partial UNIQUE index enforces.
R-OSR-RMA-2	swap_kind MUST be consistent with source/target instance equipment_kind (PLM-CFG-06). HFC_FIELD requires source AND target (when present) to be HFC-family SKUs (CABLE_MODEM, STB, SMART_CARD, MTA). GPON_* requires ONT. Eligibility worker enforces; rejects at swap creation.
R-OSR-RMA-3	EQP_TERMINATION MUST have subscription_id populated; reason_code = TERMINATION. EQP can have either a single source (one device per termination) or be paired with multiple swap-requests (one per device per subscription) — v1.0 chooses paired separate requests for clarity.
R-OSR-RMA-4	recovery_contractor_id MUST be set before current_state advances past AWAITING_SLOT. Same row, same transaction, atomic with slot_commitment_id.
R-OSR-RMA-5	When current_state advances to SOURCE_RECOVERED, the framework MUST call OSR-INSTANCE-01 POST /api/equipment-instances/{source_instance_id}/lifecycle-events with to_state=RECOVERED_BY_CONTRACTOR, contractor_id={recovery_contractor_id}, origin_ref_kind=swap_request, origin_ref_id={swap_request_id}. This is the row that triggers contractor-stock routing (the Wananchi fix).
R-OSR-RMA-6	When current_state advances to TARGET_PLACED, the framework MUST call OSR-INSTANCE-01 POST /api/equipment-instances/{target_instance_id}/lifecycle-events with to_state=IN_FIELD_ACTIVE, to_account_id={account_id}, contractor_id={recovery_contractor_id}, origin_ref_kind=swap_request.
R-OSR-RMA-7	When a swap-request is CANCELLED while slot_commitment_id IS NOT NULL, the framework MUST call EM-02 PATCH /api/contractor-slot-commitments/{id} with status=RELEASED to free the contractor's capacity. Idempotent — if the slot is already past or already released, the call is a no-op.
R-OSR-RMA-8	Slot/port preservation (HFC only — UC-PR4): for swap_kind=HFC_FIELD, the eligibility worker MUST read equipment_instance.network_slot_metadata of the source and set the same {cmts, slot, port} in the swap-request's flow_payload. The serial-rebind-broadhub-stub worker reads flow_payload.{cmts, slot, port} and includes it in the Broadhub call. Failure to preserve = FAILED with transition_reason=SLOT_PORT_MISMATCH_AT_REBIND.
R-OSR-RMA-9	flow_payload JSONB shape per kind: HFC_FIELD {cmts, slot, port, authz_bound, double_config:bool}; GPON_REMOTE {olt, frame, slot, port, remote_attempt_count}; GPON_FIELD {olt, frame, slot, port}; EQP_TERMINATION {deposit_held:numeric, deposit_refund_eligible:bool, recovery_required:bool}; EQU_UPGRADE {upgrade_from_sku, upgrade_to_sku, prorated_fee:numeric, deposit_delta:numeric}. Application layer validates per kind at create + at relevant transitions.
R-OSR-RMA-10	EQR (Equipment Recovery failed) is a sub-state reachable only from EQP_TERMINATION when the technician confirms on the WO that the customer refuses to return the equipment. Transition: FIELD_VISIT_IN_PROGRESS → COMPLETED_WITHOUT_RECOVERY. The source instance is NOT transitioned to RECOVERED_BY_CONTRACTOR; it stays in IN_FIELD_DEFECTIVE (or transitions to DECOMMISSIONED once BIL-01 writes off the deposit).

Rule	Description
R-OSR-RMA-11	Pay-first gating for chargeable swaps (Wananchi wallet funding gate). When the eligibility worker determines the swap is chargeable (warranty void / customer-caused damage / EQU upgrade fee / EQR forfeiture pre-charge), it sets <code>chargeable=TRUE</code> , resolves <code>charge_code + customer_charge_amount</code> , calls <code>BIL-01 POST /api/billable-events</code> to create the billable event (returning <code>billable_event_id</code>), and transitions the swap-request to <code>AWAITING_PAYMENT</code> . The <code>bil01-charge-pre-authorize</code> worker drives this. The swap-request does NOT advance past <code>AWAITING_PAYMENT</code> until <code>BIL-01</code> confirms the customer wallet covers the charge — confirmation arrives via the payment-confirmed Kafka message catch on the BPMN. Until then, no slot is reserved, no contractor is dispatched, no truck rolls. This codifies workshop 2026-04-28 rule "tech is dispatched only after the customer wallet is topped up". Free swaps (under warranty / system fault / Shifting-style operator-paid) set <code>chargeable=FALSE</code> and skip <code>AWAITING_PAYMENT</code> entirely.
R-OSR-RMA-11b	Payment timeout: if <code>AWAITING_PAYMENT</code> exceeds the operator-configured timeout (default 7 days), the swap-request transitions to <code>CANCELLED</code> with <code>transition_reason=PAYMENT_TIMEOUT</code> . Customer-facing reminder notifications are requested on day 1, day 3, day 6 by emitting <code>NOT-01</code> event kinds <code>OSR_RMA_PAYMENT_PENDING_D1</code> , <code>OSR_RMA_PAYMENT_PENDING_D3</code> , <code>OSR_RMA_PAYMENT_PENDING_D6</code> with the swap/account variables. <code>NOT-01</code> owns template, channel, language, and fallback. Timeout enforced by Camunda boundary timer on the <code>AWAITING_PAYMENT</code> state.
R-OSR-RMA-11c	EQU deposit delta: when <code>swap_kind=EQU_UPGRADE</code> and <code>flow_payload.deposit_delta > 0</code> , the additional deposit is part of the chargeable amount and is included in the <code>BIL-01</code> billable event at eligibility. When <code>deposit_delta < 0</code> (rare — downgrade in deposit class), the swap is NOT chargeable for the delta but a deposit-refund adjustment is queued for posting at <code>COMPLETED</code> per <code>PLM-CFG-05</code> catalog.
R-OSR-RMA-16	HomePass network_path update on GPON instance swap. When <code>swap_kind=GPON_REMOTE</code> or <code>GPON_FIELD</code> and the source instance is bound as the ONT terminal node of a HomePass (i.e., the customer's account is linked to a HomePass and the HomePass's <code>network_path</code> terminal node has <code>type=ONT</code> and <code>code={source_instance.serial_number}</code>), the framework MUST update the HomePass's <code>network_path</code> terminal node code to the new ONT serial at the <code>TARGET_PLACED</code> transition. The <code>homepass-network-path-update</code> worker drives this — it calls <code>HomePass-CFG PATCH /api/homepasses/{id}/network-path/ports</code> (for ONT-serial-only changes; if F/S/P also moves, full <code>network-path-replace</code> is used). Failure to update the HomePass = <code>FAILED</code> with <code>transition_reason=HOMEPASS_NETWORK_PATH_UPDATE_FAILED</code> . HFC swaps do NOT update HomePass (HFC equipment-to-HomePass binding is not modeled in <code>HomePass-CFG network_path</code> today). <code>EQP_TERMINATION</code> does NOT remove the ONT from HomePass — the HomePass stays at its current status; subscription closure (separate workflow) handles HomePass-side state changes.
R-OSR-RMA-12	History append-only: <code>equipment_swap_request_history</code> rows are <code>INSERT</code> only. <code>UPDATE</code> / <code>DELETE</code> rejected at the database layer. <code>event_sequence</code> is monotonic per swap-request; <code>UNIQUE</code> constraint enforces.
R-OSR-RMA-13	BPMN process instance: <code>bpmn_process_instance_id</code> is set when the BPMN starts (transition <code>DRAFT → ELIGIBILITY_CHECK</code>). On terminal state, the BPMN process completes naturally OR is force-terminated by the framework's cancellation API. Force-termination is logged in history with <code>transition_reason=BPMN_FORCE_TERMINATED</code> .
R-OSR-RMA-14	Operator scope: <code>equipment_swap_request.operator_code</code> MUST match the account's <code>operator_code</code> , the source/target instances' <code>operator_codes</code> , and the recovery contractor's <code>operator_code</code> . Cross-operator swap creation rejected.
R-OSR-RMA-15	The vendor RMA stub link is optional; not every recovered defective instance goes to vendor (some are repaired in-house, some are write-offs). When <code>outbound_rma_stub_id</code> is set on the swap-request, the source instance's lifecycle MUST transition to <code>RMA_SHIPPED_TO_VENDOR</code> no later than the swap-request's <code>COMPLETED</code> transition. Application layer enforces via outbox.

5. Shared Camunda primitives

Per `FOUNDATION_CAMUNDA`, all workers are operator-agnostic Java beans registered with the Camunda engine. They take input via process variables, perform their action, write output process variables, and complete the external task.

5.1 Service-task workers (14 shared workers)

Worker name	Purpose	Inputs (process variables)	Outputs			
instance-lookup	Resolve an instance from any identifier (instance_id, serial, MAC, smart card)	lookupKey (str), lookupKind ('instance_id')	'serial'	'mac'	'smart_card')	instance (JSON), currentState, currentLocationId, currentAccountId
instance-state-transition	Call OSR-INSTANCE-01 to advance an instance through one lifecycle event	instanceId, toState, toLocationId?, toAccountId?, toSentinel?, reasonCode, contractorId?, originRefKind, originRefId, notes?	lifecycleEventId, eventSequence			
eligibility-check	Validate swap eligibility + resolve chargeable status + charge_code	accountId, subscriptionId, sourceInstanceId, targetInstanceId?, swapKind, reasonCode	eligible (bool), failureCode?, chargeable (bool), chargeCode?, customerChargeAmount?, flowPayload (JSONB)			
bil01-charge-prere-authorize	Pay-first gate worker. When chargeable=TRUE, call BIL-01 POST /api/billable-events to create a billable event for the customer's wallet; returns the billable_event_id. The BPMN then transitions to AWAITING_PAYMENT and waits for the payment-confirmed message catch (raised when BIL-01 emits the matching BillableEventPaid event for this billable_event_id).	swapRequestId, accountId, chargeCode, customerChargeAmount, flowPayload (for charge context: upgrade_to_sku, deposit_delta, etc.)	billableEventId, chargeCreatedAt			
contractor-slot-reserve	Reserve a slot against EM-02 contractor availability	accountId, techRegionId, serviceScope, requiredSkills (list), windowStart, windowEnd	contractorId, slotCommitmentId			
contractor-slot-release	Release a previously-reserved slot	slotCommitmentId	(no output)			
contractor-stock-route	When a unit is recovered, route it to the recovering contractor's warehouse (the WANANCHI FIX)	instanceId, recoveryContractorId, recoveryEventId	stockMovementId, targetLocationId			
serial-unbind-broadhub-stub	Call Broadhub provisioning adapter to unbind a CM-MAC / MTA-MAC; HFC-only	instanceId, cmMac, mtaMac?, cpMac?, slotPort (JSON)	adapterResponseCode, unbindOk (bool)			
serial-rebind-broadhub-stub	Call Broadhub adapter to bind the new CM-MAC at the same slot/port (UC-PR4 preservation)	instanceId, cmMac, mtaMac?, cpMac?, slotPort (JSON, MUST match source's slotPort)	adapterResponseCode, rebindOk (bool)			

Worker name	Purpose	Inputs (process variables)	Outputs			
nms-remote-unassign-stub	Call NMS adapter to unassign a GPON ONT (remote operation)	instanceId, ontSerial, oltFrameSlotPort (JSON)	adapterResponseCode, unassignOk (bool)			
nms-remote-bind-stub	Call NMS adapter to bind a new GPON ONT remotely	instanceId, ontSerial, oltFrameSlotPort (JSON)	adapterResponseCode, bindOk (bool)			
homepass-network-path-update	HomePass terminal-node sync for GPON swaps. When the source GPON ONT is bound to a HomePass terminal node (HomePass.network_path[i] with type=ONT and code=source_serial), update the node's code to the new ONT serial. Calls HomePass-CFG PATCH /api/homepasses/{id}/network-path/ports for serial-only changes; if F/S/P also changes, calls PATCH /api/homepasses/{id}/network-path (full path replace). Resolves the HomePass via the account's homepass_id binding (ILM-CFG-01 → HomePass-CFG cross-reference).	accountId, sourceInstanceId, targetInstanceId, swapKind	homepassId?, networkPathUpdated (bool), previousOntSerial?, newOntSerial?			
swap-request-history-append	Append a history row	swapRequestId, fromState, toState, transitionReason, actorUserId, workerName?, payload?	historyId, eventSequence			
notification-send	Request a notification via NOT-01 (customer) or ICN-01 (internal)	recipientKind ('customer')	'group', recipientRef (account_id or group_code), eventKind, notificationVars (JSON), correlationRef	notificationRequestId, accepted (bool)		

Worker registration:

```
# camunda-bpm-config/external-task-config.yaml (operator-agnostic)
external-tasks:
  - topic: instance-lookup
    handler-class: com.sophix.osr.rma.workers.InstanceLookupHandler
    lock-duration-ms: 30000
    max-tasks: 10
  - topic: instance-state-transition
    handler-class: com.sophix.osr.rma.workers.InstanceStateTransitionHandler
    lock-duration-ms: 60000
    max-tasks: 10
  - topic: eligibility-check
    handler-class: com.sophix.osr.rma.workers.EligibilityCheckHandler
    lock-duration-ms: 120000
    max-tasks: 5
  - topic: contractor-slot-reserve
    handler-class: com.sophix.osr.rma.workers.ContractorSlotReserveHandler
    lock-duration-ms: 60000
```

```

    max-tasks: 5
- topic: contractor-slot-release
  handler-class: com.sophix.osr.rma.workers.ContractorSlotReleaseHandler
  lock-duration-ms: 30000
  max-tasks: 5
- topic: contractor-stock-route
  handler-class: com.sophix.osr.rma.workers.ContractorStockRouteHandler
  lock-duration-ms: 60000
  max-tasks: 5
- topic: serial-unbind-broadhub-stub
  handler-class: com.sophix.osr.rma.workers.SerialUnbindBroadhubStubHandler
  lock-duration-ms: 120000
  max-tasks: 3
- topic: serial-rebind-broadhub-stub
  handler-class: com.sophix.osr.rma.workers.SerialRebindBroadhubStubHandler
  lock-duration-ms: 120000
  max-tasks: 3
- topic: nms-remote-unassign-stub
  handler-class: com.sophix.osr.rma.workers.NmsRemoteUnassignStubHandler
  lock-duration-ms: 120000
  max-tasks: 3
- topic: nms-remote-bind-stub
  handler-class: com.sophix.osr.rma.workers.NmsRemoteBindStubHandler
  lock-duration-ms: 120000
  max-tasks: 3
- topic: bil01-charge-pre-authorize
  handler-class: com.sophix.osr.rma.workers.Bil01ChargePreAuthorizeHandler
  lock-duration-ms: 60000
  max-tasks: 5
- topic: homepass-network-path-update
  handler-class: com.sophix.osr.rma.workers.HomepassNetworkPathUpdateHandler
  lock-duration-ms: 60000
  max-tasks: 5
- topic: swap-request-history-append
  handler-class: com.sophix.osr.rma.workers.SwapRequestHistoryAppendHandler
  lock-duration-ms: 30000
  max-tasks: 10
- topic: notification-send
  handler-class: com.sophix.osr.rma.workers.NotificationSendHandler
  lock-duration-ms: 30000
  max-tasks: 10

```

5.2 Concrete worker call example — instance-state-transition

Camunda BPMN service task uses HTTP connector to invoke an external worker via `fetchAndLock` + `complete`:

```

POST /engine-rest/external-task/fetchAndLock HTTP/1.1
Content-Type: application/json

{
  "workerId": "osr-rma-worker-pod-03",
  "maxTasks": 1,
  "topics": [
    { "topicName": "instance-state-transition", "lockDuration": 60000 }
  ]
}

→ 200 OK
[
  {
    "id": "extt_01HZ_inst_state",
    "topicName": "instance-state-transition",
    "processInstanceId": "pi_01HZ_NJERI_SWAP",
    "variables": {
      "instanceId": { "value": "inst_01HZ_CM_NJERI_OLD", "type": "String" },
      "toState": { "value": "RECOVERED_BY_CONTRACTOR", "type": "String" },
      "toLocationId": { "value": "WIK-VAN-contractor_ke_nairobi_a", "type": "String" },
      "reasonCode": { "value": "CONTRACTOR_RECOVERED_FROM_FIELD", "type": "String" },
      "contractorId": { "value": "contractor_ke_nairobi_a", "type": "String" },
      "originRefKind": { "value": "swap_request", "type": "String" },
      "originRefId": { "value": "swap_01HZ_NJERI_MODEM", "type": "String" }
    }
  }
]

```

The worker pod then calls `OSR-INSTANCE-01`:

```

POST /api/equipment-instances/inst_01HZ_CM_NJERI_OLD/lifecycle-events HTTP/1.1
Authorization: Bearer <service-jwt>
Content-Type: application/json

{

```

```

    "toState": "RECOVERED_BY_CONTRACTOR",
    "toLocationId": "WIK-VAN-contractor_ke_nairobi_a",
    "fromAccountId": "acct_01HZ_NJERI",
    "reasonCode": "CONTRACTOR_RECOVERED_FROM_FIELD",
    "contractorId": "contractor_ke_nairobi_a",
    "originRefKind": "swap_request",
    "originRefId": "swap_01HZ_NJERI_MODEM",
    "actorUserId": "tech_franchise_a_03",
    "expectedEventSequence": 5
  }
}

→ 201 Created
{
  "eventId": "evt_01HZ_NJERI_RECOVERED",
  "eventSequence": 6
}

```

Worker completes the Camunda task:

```

POST /engine-rest/external-task/extt_01HZ_inst_state/complete HTTP/1.1
Content-Type: application/json

{
  "workerId": "osr-rma-worker-pod-03",
  "variables": {
    "lifecycleEventId": { "value": "evt_01HZ_NJERI_RECOVERED", "type": "String" },
    "eventSequence": { "value": 6, "type": "Long" }
  }
}

→ 204 No Content

```

5.3 Shared messages (Camunda message catch events)

Message name	Sent by	Caught by (satellite BPMNs)	Carries
wo-finalized	WO-01 on WO completion	All field-visit flows (SWAP-HFC, SWAP-GPON-FIELD, EQP, EQU)	woId, outcome ('SUCCESS'/EQR_FAILED_RETURN'/CUSTOMER_REFUSED_APPOINTMENT'), recoveredSerial?, placedSerial?
payment-confirmed	BIL-01 (Kafka event BillableEventPaid correlated by billable_event_id)	All chargeable flows (after AWAITING_PAYMENT)	billableEventId, paidAt, walletId, paidAmount
payment-timeout	Camunda boundary timer on AWAITING_PAYMENT user task (operator-configured, default 7 days)	All chargeable flows	(no payload)
customer-appointment-accepted	NOT-01 inbound webhook (customer SMS reply)	All field-visit flows	swapRequestId, appointmentSlot
customer-appointment-rejected	NOT-01 inbound webhook	All field-visit flows	swapRequestId, rejectionReason?
remote-attempt-timeout	Timer boundary on REMOTE_UNASSIGN_IN_PROGRESS user task	SWAP-GPON-REMOTE only	(no payload)
nms-remote-completed	nms-remote-unassign-stub worker on success	SWAP-GPON-REMOTE only	unassignOk (bool), adapterResponseCode
bpmn-cancel-requested	API: POST /api/swap-requests/{id}/cancel	All flows	reason, actorUserId

5.4 Shared user tasks (Camunda user tasks)

User task ID	Candidate group	Form fields	Used by	
swap-dispatch-slot-selection	swap-dispatchers (FOUNDATION_AUTH)	selectedContractorId, selectedSlotId, appointmentDateTime, notes?	All field-visit flows (after AWAITING_SLOT)	
tech-confirm-defect	field-techs	defectConfirmed (bool), defectNotes (str)	SWAP-HFC, SWAP-GPON-FIELD (on-site verification step)	
tech-confirm-recovery-or-eqr	field-techs	recoveryOutcome ('RECOVERED')	'CUSTOMER_REFUSED_RETURN', notes?	EQP only
noc-confirm-remote-success	noc-team	customerSelfSwapConfirmed (bool), notes?	SWAP-GPON-REMOTE happy-path	

User task ID	Candidate group	Form fields	Used by	
warranty-approver-decision	finance-warranty-approvers	chargeApproved (bool), chargeCode?, notes	All flows on eligibility-uncertain path (e.g., warranty status unclear)	
vendor-rma-coordinator-batch-ship	vendor-rma-coordinators	vendorName, vendorRef?, batchInstanceIds (list), notes	Future expansion — placeholder declared here	

5.5 Shared Drools rules

```

package com.sophix.osr.rma.rules;

import com.sophix.osr.rma.facts.SwapEligibilityRequest;
import com.sophix.osr.rma.facts.RoutingDecisionRequest;

rule "swap-eligibility-account-must-be-active"
when
    $req: SwapEligibilityRequest($req.accountStatus != "ACTIVE" && $req.swapKind != "EQP_TERMINATION")
then
    $req.setEligible(false);
    $req.setFailureCode("ACCOUNT_NOT_ACTIVE");
end

rule "swap-eligibility-source-instance-must-be-in-field"
when
    $req: SwapEligibilityRequest($req.sourceInstanceCurrentState not in ("IN_FIELD_ACTIVE", "IN_FIELD_DEFECTIVE"))
then
    $req.setEligible(false);
    $req.setFailureCode("SOURCE_INSTANCE_NOT_IN_FIELD");
end

rule "swap-eligibility-hfc-target-must-match-equipment-kind"
when
    $req: SwapEligibilityRequest($req.swapKind == "HFC_FIELD" && $req.sourceEquipmentKind != $req.targetEquipmentKind)
then
    $req.setEligible(false);
    $req.setFailureCode("EQUIPMENT_KIND_MISMATCH");
end

rule "routing-decision-recovered-instance-routes-to-recovering-contractor"
when
    $req: RoutingDecisionRequest($req.transitionReason == "CONTRACTOR_RECOVERED_FROM_FIELD")
then
    $req.setTargetLocationKind("CONTRACTOR_VAN");
    $req.setTargetContractorId($req.recoveryContractorId);
end

rule "routing-decision-main-warehouse-fallback"
when
    $req: RoutingDecisionRequest($req.transitionReason == "CONTRACTOR_RECOVERED_FROM_FIELD" && $req.recoveryContractorId == null)
then
    $req.setTargetLocationKind("MAIN_WAREHOUSE");
    $req.setWarning("RECOVERY_CONTRACTOR_MISSING_FALLBACK_MAIN");
end

```

The "routing-decision-recovered-instance-routes-to-recovering-contractor" rule is the formal codification of the Wananchi 40+ stale-equipment fix: a recovered unit's home is its recovering contractor's van, NOT main warehouse. The fallback rule fires only when `recovery_contractor_id` is missing (data integrity edge case) and emits a warning so ops can investigate.

5.6 FOUNDATION_AUTH groups consumed

Group code	Owned by	Used as candidateGroup in
swap-dispatchers	FOUNDATION_AUTH	swap-dispatch-slot-selection user task
field-techs	FOUNDATION_AUTH (often combined with tech-region-<region> for routing)	tech-confirm-defect, tech-confirm-recovery-or-eqr
noc-team	FOUNDATION_AUTH	noc-confirm-remote-success
vendor-rma-coordinators	FOUNDATION_AUTH	vendor-rma-coordinator-batch-ship (future)
finance-warranty-approvers	FOUNDATION_AUTH	warranty-approver-decision

In practice, satellite BPMNs often combine these with tech-region scoping: `candidateGroups="field-techs, tech-region-NRB-KAREN"` resolves via ICN-01 to the intersection — only field techs in the NRB-KAREN region.

6. REST API (framework-level)

The framework exposes only the base swap-request endpoints. Each satellite extends with its own create-shape that pre-populates the discriminator + flow_payload.

6.1 Base swap-request CRUD (framework)

```
GET /api/swap-requests/{swap_request_id}
GET /api/swap-requests?operator=WIK&accountId=acct_01HZ_NJERI # all swaps for one account
GET /api/swap-requests?operator=WIK&swapKind=HFC_FIELD&currentState=AWAITING_SLOT # open HFC swaps awaiting dispatch
GET /api/swap-requests?operator=WIK&recoveryContractorId=contractor_ke_nairobi_a # all swaps assigned to a contractor
GET /api/swap-requests?operator=WIK&currentState=FAILED&since=2026-03-01 # failed swaps for ops review
GET /api/swap-requests/{swap_request_id}/history # full history ledger
```

6.2 Framework-level state operations

```
POST /api/swap-requests/{swap_request_id}/advance HTTP/1.1 # used by BPMN workers + back-office UI
Authorization: Bearer <service-jwt>
Content-Type: application/json
```

```
{
  "toState": "SOURCE_RECOVERED",
  "transitionReason": "TECH_RECOVERED_SOURCE",
  "actorUserId": "tech_franchise_a_03",
  "workerName": "instance-state-transition",
  "payload": { "lifecycleEventId": "evt_01HZ_NJERI_RECOVERED", "eventSequence": 6 }
}
```

→ 200 OK

```
{
  "swapRequestId": "swap_01HZ_NJERI_MODEM",
  "currentState": "SOURCE_RECOVERED",
  "historyId": "swh_01HZ_E7",
  "eventSequence": 7
}
```

```
POST /api/swap-requests/{swap_request_id}/cancel HTTP/1.1
Authorization: Bearer <bo-user-jwt>
Content-Type: application/json
```

```
{
  "reason": "Customer cancelled – no longer experiencing the issue",
  "actorUserId": "support_agent_wik_02"
}
```

→ 200 OK

```
{
  "swapRequestId": "swap_01HZ_PROACTIVE_BATCH_05",
  "currentState": "CANCELLED",
  "finalState": "CANCELLED",
  "slotReleased": true,
  "woCancelled": false
}
```

Cancellation behavior:

- If slot_commitment_id IS NOT NULL, calls EM-02 PATCH /api/contractor-slot-commitments/{id} with status=RELEASED.
- If wo_id IS NOT NULL AND wo.current_state != COMPLETED, calls WO-01 PATCH /api/work-orders/{id} with status=CANCELLED.
- If bpmn_process_instance_id IS NOT NULL, calls Camunda DELETE /engine-rest/process-instance/{id} to force-terminate.

6.3 Vendor RMA stub CRUD

```
POST /api/vendor-rma-stubs HTTP/1.1
{
  "operatorCode": "WIK",
  "vendorName": "Huawei",
  "vendorRef": "HUA-RMA-2026-0234",
  "shippedAt": "2026-03-15T10:00:00+03:00",
  "expectedReturnAt": "2026-04-15",
  "instanceIds": [
    "inst_01HZ_ONT_DEFECT_01",
  ]
}
```

```

    "inst_01HZ_ONT_DEFECT_02"
  ],
  "initiatedByUserId": "vendor-rma-coord-wik-01",
  "notes": "Batch of 2 Huawei ONTs flagged by Franchise A as beyond field-repair"
}

→ 201 Created
{
  "stubId": "rma_01HZ_BATCH_HUAWEI_05",
  "vendorRef": "HUA-RMA-2026-0234",
  "linkedInstances": 2
}

```

Linking the instances triggers lifecycle transitions on each: RETURNED_TO_MAIN_WAREHOUSE → RMA_SHIPPED_TO_VENDOR per OSR-INSTANCE-01 §4.1.

```

PATCH /api/vendor-rma-stubs/{stub_id} HTTP/1.1
{
  "returnedAt": "2026-03-25T09:00:00+03:00",
  "returnOutcome": "REPAIRED",
  "vendorNotes": "All 2 units repaired; firmware reflashed to v2.34"
}

```

Marking the stub returned triggers lifecycle transitions: RMA_SHIPPED_TO_VENDOR → REPAIRED_BY_VENDOR (or → DECOMMISSIONED when returnOutcome=BEYOND_REPAIR).

7. Cross-DD coordination

7.1 Upstream callers (write paths)

Caller	Endpoint	When
NOC / customer support	Satellite-specific POST /api/swap-requests/...	Customer reports defect or upgrade request
Network monitoring (future)	Satellite-specific POST /api/swap-requests/... with reason_code=SYSTEM_FAULT_PROACTIVE	Automated detection of failing devices
Subscription termination flow (SUB-WF-TERMINATE-01)	EQP satellite POST /api/swap-requests/eqp	Subscription closes; equipment retrieval needed
BPMN workers	POST /api/swap-requests/{id}/advance	Throughout each flow
Back-office support agents	POST /api/swap-requests/{id}/cancel	Manual cancellation
Vendor RMA coordinators	POST /api/vendor-rma-stubs, PATCH /api/vendor-rma-stubs/{id}	Ship batch / receive return

7.2 Downstream consumers (read paths + events)

Topic prefix: `sophix.osr.rma.*` (events emitted by satellites; framework does not emit events directly but provides the infrastructure).

Satellites emit:

Event	Topic	Fires on
SwapRequestCreated	<code>sophix.osr.rma.created</code>	DRAFT → ELIGIBILITY_CHECK
SwapRequestAwaitingPayment	<code>sophix.osr.rma.awaiting-payment</code>	ELIGIBILITY_CHECK → AWAITING_PAYMENT (chargeable swaps only)
SwapRequestPaymentConfirmed	<code>sophix.osr.rma.payment-confirmed</code>	AWAITING_PAYMENT → AWAITING_SLOT
SwapRequestSourceRecovered	<code>sophix.osr.rma.source-recovered</code>	* → SOURCE_RECOVERED
SwapRequestHomePassUpdated	<code>sophix.osr.rma.homepass-updated</code>	After homepass-network-path-update worker succeeds (GPON swaps only)
SwapRequestCompleted	<code>sophix.osr.rma.completed</code>	* → COMPLETED
SwapRequestCompletedWithoutRecovery	<code>sophix.osr.rma.completed-without-recovery</code>	* → COMPLETED_WITHOUT_RECOVERY (EQR sub-state)
SwapRequestCancelled	<code>sophix.osr.rma.cancelled</code>	* → CANCELLED (includes payment-timeout path)
SwapRequestFailed	<code>sophix.osr.rma.failed</code>	* → FAILED

Consumers:

Consumer	Events subscribed	Action
OSR-INSTANCE-01	(consumed indirectly via worker calls; doesn't subscribe)	Updates per-instance state in same transaction
OSR-01 Stock Chain Engine	(consumed indirectly via contractor-stock-route worker; doesn't subscribe)	Records stock_movement for each recovered/placed unit
BIL-01 Charging Engine	(called upstream at AWAITING_PAYMENT via worker bil01-charge-pre-authorize; emits BillableEventPaid consumed by message catch payment-confirmed) — also subscribes SwapRequestCompletedWithoutRecovery for deposit forfeiture	Pay-first charge creation at AWAITING_PAYMENT; deposit forfeiture on EQR
BIL-02 Invoicing	(via BIL-01)	Charged amounts appear on next invoice cycle
HomePass-CFG	(consumed indirectly via worker homepass-network-path-update; doesn't subscribe)	Receives PATCH /api/homepasses/{id}/network-path/... on TARGET_PLACED for GPON swaps
NOT-01	OSR_RMA_PAYMENT_PENDING_D1, OSR_RMA_PAYMENT_PENDING_D3, OSR_RMA_PAYMENT_PENDING_D6, OSR_RMA_CREATED, OSR_RMA_COMPLETED, OSR_RMA_FAILED, OSR_RMA_COMPLETED_WITHOUT_RECOVERY	Customer notifications; NOT-01 owns template, channel, language, and fallback.
ICN-01	All sophix.osr.rma.*	Internal notification to swap-dispatchers, field-techs, noc-team, finance-warranty-approvers
WO-01	SwapRequestCancelled	If a WO is open for this swap, cancel it
EM-04 Reporting (pending)	All sophix.osr.rma.*	Metrics: mean time to recover, per-contractor swap volumes, swap failure rates, EQR rate, charge realization rate, payment-timeout cancellation rate

7.3 Sibling modules

Module	Direction	Contract
OSR-INSTANCE-01	OSR-RMA-01 → OSR-INSTANCE-01 (write)	Every state transition that changes an instance calls OSR-INSTANCE-01 lifecycle event API
OSR-01 Stock Chain Engine	OSR-RMA-01 → OSR-01 (write, via worker)	contractor-stock-route worker writes stock_movement rows for SKU+qty rollup
EM-02 Contractor & Staff Registry	OSR-RMA-01 → EM-02 (read+write)	Read: contractor availability lookup. Write: slot reservations + releases
WO-01 Work Order Module	OSR-RMA-01 → WO-01 (write)	Create WO for field-visit flows; receive wo-finalized message back
ILM-CFG-01	OSR-RMA-01 → ILM-CFG-01 (read)	Read account.tech_region_id for contractor selection; read subscription for EQP
BIL-01 Charging Engine	OSR-RMA-01 ↔ BIL-01 (write upstream + message back)	Pay-first gate: at AWAITING_PAYMENT worker bil01-charge-pre-authorize POSTs billable event; BIL-01 emits BillableEventPaid Kafka event correlated by billable_event_id; BPMN message catch payment-confirmed advances to AWAITING_SLOT. Also: read warranty/charge-code catalog at eligibility; write deposit forfeiture on EQR.
HomePass-CFG	OSR-RMA-01 → HomePass-CFG (write)	GPON swaps only: at TARGET_PLACED, homepass-network-path-update worker calls PATCH /api/homepasses/{id}/network-path/ports to update the terminal ONT node's code to the new ONT serial. Resolves HomePass via account.homepass_id (ILM-CFG-01 binding).
NOT-01	OSR-RMA-01 → NOT-01 (write)	Customer notifications including payment-pending reminders (d1/d3/d6)
ICN-01	OSR-RMA-01 → ICN-01 (write)	Internal notifications
FOUNDATION_AUTH	OSR-RMA-01 → FOUNDATION_AUTH (read)	Group membership resolution for user tasks
FOUNDATION_CAMUNDA	(deploys to)	Workers, BPMNs, Drools KJARs live in Camunda
PLM-CFG-06	OSR-RMA-01 → PLM-CFG-06 (read)	Equipment SKU + equipment_kind for eligibility validation

7.4 Build order

OSR-RMA-01-FRAMEWORK deploys after:

1. PLM-CFG-06 (equipment_sku for FK chains)
2. OSR-01 (stock_location, stock_movement)
3. OSR-INSTANCE-01 (equipment_instance, lifecycle event ledger)
4. EM-02 (contractor + contractor_slot_commitment)
5. WO-01 (wo table; for the satellite flows that create WOs)
6. ILM-CFG-01 (account + subscription; account.homepass_id binding for HomePass resolution)
7. HomePass-CFG (network_path PATCH endpoint for GPON swap terminal-node update)
8. BIL-01 (POST /api/billable-events + BillableEventPaid Kafka event for pay-first gate)
7. NOT-01 + ICN-01 (notification fanouts)
8. BIL-01 (charging triggers)
9. FOUNDATION_AUTH (group codes)
10. FOUNDATION_CAMUNDA (engine)

Satellites (SWAP-HFC, SWAP-GPON, EQP, EQU) deploy after this framework.

8. Workshop sources

Workshop sources for v1.0 (KE Zuku 2026-04-27+; no fabricated TZ baselines — TBD pending cartography):

- **BP-08 Equipment Swap (RMA) GAP analysis** — technology-divergent swap mechanics (HFC = field pickup, GPON = remote unassign + new bind), device authorization slot/port mandatory (UC-PR4), serial unbind/rebind (UC-PR9), contractor-stock traceability gap (default routes to main warehouse — broken; documented as 40+ stale items). G-08-1 (workflow) + G-08-2 (contractor stock traceability) — both addressed by this framework + satellites.
- **Workshop 2026-04-30 (Provisioning deep dive)** — HFC modem authorization bound to slot/port (UC-PR4 KE-specific); static-IP HFC double config (Broadhub + Clearcable NOMS); static-IP GPON manual via DHCP console; voice MTA/IAD ports.
- **Workshop 2026-04-30 (Customer Management deep dive)** — Equipment-routing gap: unassigned equipment auto-returns to main warehouse instead of recovering contractor team's warehouse (~40+ stale items). Wananchi target: SOPHIX must trace contractor team that recovered + route there. THIS FRAMEWORK'S recovery_contractor_id on equipment_swap_request + the routing-decision-recovered-instance-routes-to-recovering-contractor Drools rule + the contractor-stock-route worker collectively close this gap.
- **Workshop 2026-04-29 (Work Order & Field Operations deep dive)** — WO Job Types EQU (Equipment Upgrade), EQP (Equipment Pickup), EQR (Equipment Recovery — closes EQP when customer paid for CPE & refuses return). These map to the four satellites' BPMN kinds.
- **Workshop 2026-04-30 (Service Termination)** — Equipment dismantle is a separate retrieval WO if customer doesn't bring CPE back. Agent contractor dispatched. Refund processed via Invoice Adjustments (BIL-01 + PLM-CFG-05 adjustment codes). Drives EQP satellite + EQR sub-state.

TZ codebase coverage today: to be filled in after codebase review. No equivalent of multi-state swap-request workflow has been confirmed in the TZ codebase yet; assessment pending cartography.

Gap & V3 design rationale: The OSR-RMA-01 family introduces an explicit Camunda-orchestrated swap workflow (replacing ad-hoc ticket-revamp + on-site action flow) with per-tech BPMN variants, formal slot/port preservation for HFC, formal contractor-stock-routing for recovered units (closing the documented 40+ stale-equipment gap), and an outbound-RMA stub for future expansion.